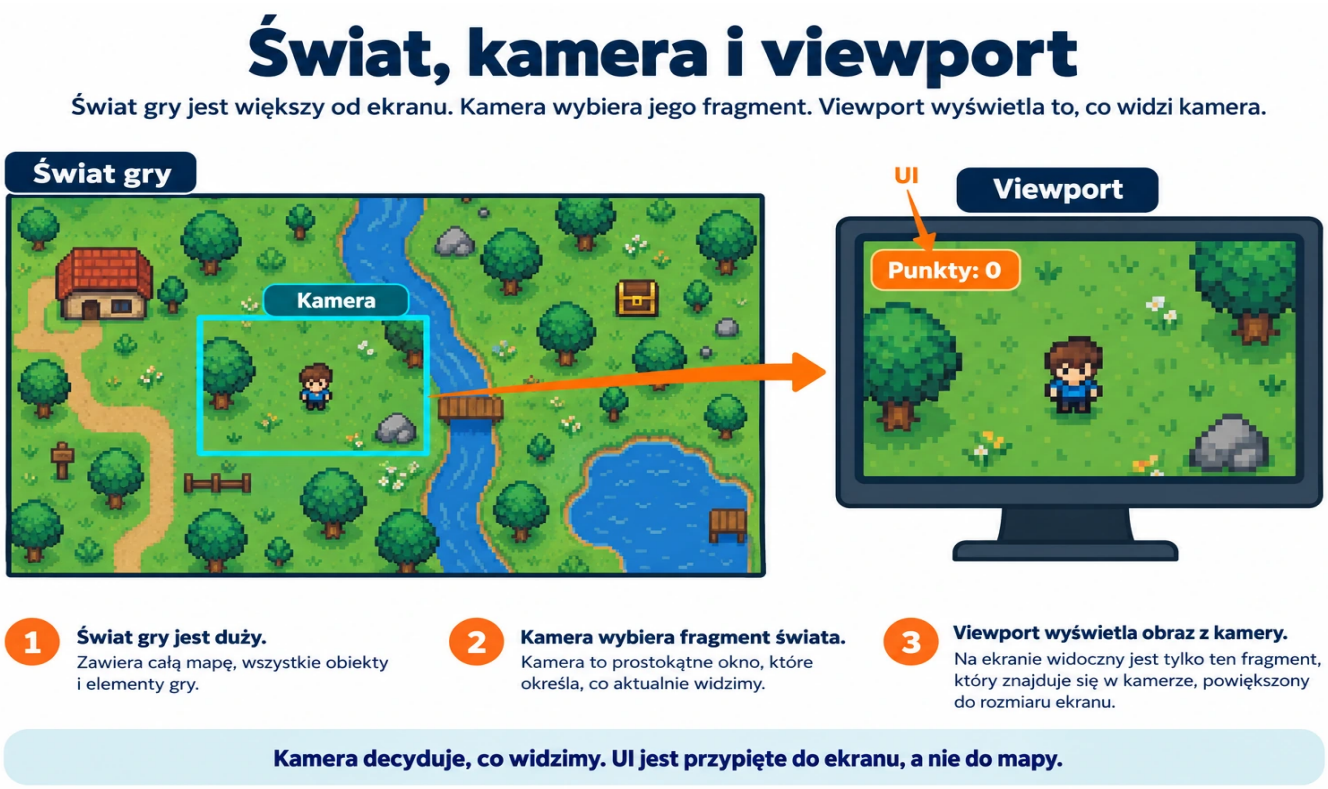
Świat, kamera i viewport



Świat zawiera pozycje obiektów. **Kamera** wybiera widoczny fragment świata. **Viewport** jest obszarem planszy canvas, nie całym oknem przeglądarki.

Ilustracja pokazuje ogólną ideę; obecne API Java Camera2D przesuwa widok, ale nie udostępnia zoomu. Powiększenie canvasu przez CSS nie jest zmianą pozycji obiektów ani fizyki.

HUD to tekst lub inne elementy przypięte do ekranu. Gdy kamera przesuwa mapę, licznik punktów powinien pozostać w tym samym miejscu.

Śledzenie gracza

Dodaj te instrukcje do onCreate() po przygotowaniu gracza z poprzedniego kursu. Nazwa player wskazuje już istniejący obiekt, nie nową kopię postaci.

```
GameObject cameraObject = createObject("Camera");
Camera2D camera = cameraObject.addComponent(new Camera2D());
camera.follow(player);
CharacterController2D controller =
    player.getComponent(CharacterController2D.class);
controller.constrainToBounds = false;
```

constrainToBounds = false pozwala wyjść poza początkowy obszar canvasu. Własne ściany i granice mapy przygotowujesz osobno. Java używa tej nazwy pola; nie kopiuj innej nazwy z kodu JavaScript.

Przelicz pozycję

Bez zoomu i drżenia: ekran = świat − przesunięcie widoku. Gdy Player jest w (500, 300), a canvas ma 640 × 360, follow ustawia widok tak, aby Player był w (320, 180) na ekranie.

```
Vector2 world = new Vector2(700, 300);
Vector2 screen = camera.worldToScreen(world);
// Dla przykładu powyżej: screen = (520, 180).
```

Offset i znaczniki świata

Kontynuuj onCreate() z poprzedniej strony. Samo przestawienie Transform obiektu Camera nie steruje widokiem: używaj follow(), stopFollowing() i offsetX/offsetY.

```
camera.stopFollowing();
camera.offsetX = 180;
camera.offsetY = 120;
// Powrót do śledzenia bez dodatkowego przesunięcia:
camera.offsetX = 0;
camera.offsetY = 0;
camera.follow(player);
camera.shake(4, 0.2);
```

Przy śledzeniu offset jest dodawany do przesunięcia widoku. Dodatni offsetX przesuwa widok w prawo, więc obiekty wyglądają na przesunięte w lewo. Shake drży obrazem, bez przesuwania fizycznego gracza.

```
GameObject ground = createObject("Ground");
ground.addComponent(new TileMap(Assets.GRASS, 32));
GameObject marker = createObject("Marker");
marker.setPosition(700, 300);
marker.addComponent(new Sprite(Assets.CHEST, 32, 32));
```

TileMap tworzy powtarzalne tło, a nie gotową mapę z kolizjami. Przy Player w (500, 300), canvasie 640 × 360 i follow(), Marker w (700, 300) jest na ekranie w (520, 180). OffsetX = 40 zmienia ekranowe X na 480.

TextRenderer w przestrzeni ekranu

Poniższy pełny przykład zapisujesz w GameMain.java. W Javie tekst aktualizujesz przez pole score.text. Nie przekazujesz funkcji JavaScript do konstruktora TextRenderer.

```
import engine.*;
public class GameMain extends Game {
    private int points = 0;
    private TextRenderer score;
    @Override public void onCreate() {
        GameObject hud = createObject("HUD");
        hud.setPosition(12, 24);
        score = hud.addComponent(
            new TextRenderer("Punkty: 0", 18, "#ffffff")
        );
        score.space = "screen";
        score.align = "left";
        score.baseline = "alphabetic";
    }
    @Override public void onUpdate(double delta) {
        if (input.isKeyPressed("Space")) points++;
        score.text = "Punkty: " + points;
    }
}
```

space = "screen" odłącza tekst od ruchu kamery. **align = "left"** przypina lewy brzeg tekstu do X = 12. Przy baseline = "alphabetic" Y = 24 oznacza linię bazową tekstu.

isKeyPressed reaguje raz na nowe naciśnięcie. isKeyDown oznacza przytrzymanie i dodawałby punkty wielokrotnie. Własny komponent używa getGame().input zamiast input sceny.

Mapa się przesuwa, HUD zostaje

Połącz gracza WASD z kamerą śledzącą, tłem i HUD-em. Dodaj trzy skrzynki o rozmiarze 32 × 32 px w pozycjach (100, 100), (700, 300) i (900, 600). Wyłącz constrainToBounds w kontrolerze gracza.

Kryteria zaliczenia

1. Przy offsetach 0 gracz jest w środku widoku.
2. Ruch kamery nie zmienia pozycji skrzynek w świecie.
3. HUD pozostaje przy ekranie.
4. Nowe naciśnięcie spacji dodaje jeden punkt; przytrzymanie nie dodaje go co klatkę.
5. Możesz wyjść poza szerokość początkowego canvasu.

Sprawdź i wyjaśnij

Czy obiekt poza ekranem znika? **Nie.**
Czy kamera przesuwa collidery w świecie? **Nie.**
Czy setPosition() obiektu Camera zastępuje follow()? **Nie.**
Czy API Camera2D ma zoom? **Nie w tej wersji.**
Jak HUD pozostaje na ekranie? **TextRenderer.space = "screen".**

Wariant: zamiast zwykłej pozycji HUD użyj UITransform i kotwic z dokumentacji silnika. Wprowadzaj jedną zmianę naraz, aby oddzielić błędy świata, kamery i interfejsu.